

LangGraph e Orquestração Baseada em Grafos

Agentes de IA – Grupo G1 | Cecília Rufatto, Felipe Proença, Leonardo Moino, Leonardo Quartaroli, Lorena Scabello

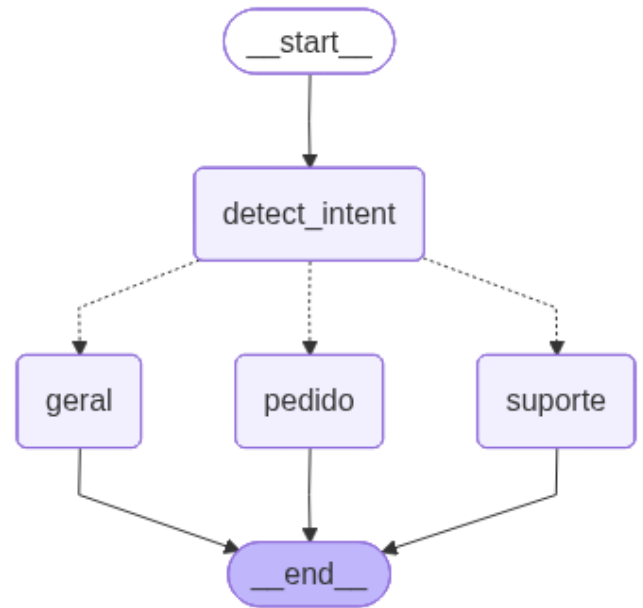
Contexto

Frameworks lineares como o LangChain tradicional executam etapas em ordem fixa, sem suporte nativo a ciclos, estado persistente ou decisões condicionais. O LangGraph cobre essa lacuna tratando o agente como um grafo de estados: nós são funções sobre o estado, arestas decidem o caminho e *checkpoints* oficiais cuidam da persistência.

Conceitos-chave

- **StateGraph** — grafo cujo estado é um `TypedDict` compartilhado entre nós.
- **Node** — função Python que recebe e devolve o estado.
- **Conditional Edge** — roteamento dinâmico via `router(state)`.
- **Reducer** — estratégia de *merge* para campos do estado.
- **Checkpoint** — persistência entre execuções (SQLite, Postgres, Redis).
- **HITL** — pausa via `interrupt` e retoma com `Command`.

Arquitetura do protótipo

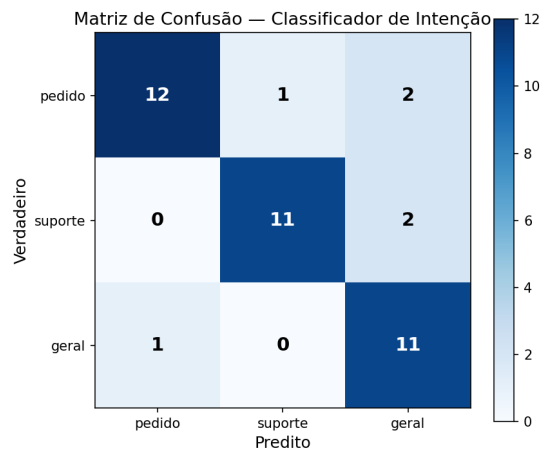


Cada mensagem é processada pelo `detect_intent`, rotulada como *pedido*, *suporte* ou *geral*, e despachada pela aresta condicional `route_intent` pro handler. Cada handler grava o histórico em JSON pelo `memory.py`.

Resultados em um dataset de 40 mensagens (pt-BR)

Métrica	Valor
Accuracy	85,00%
Macro F1	0,8507
Latência média	33,52 ms
Latência P95	~56 ms

Classe	P	R	F1
pedido	0,92	0,80	0,86
suporte	0,92	0,85	0,88
geral	0,73	0,92	0,81



Estado da arte

Linha do tempo

- **2022** – LangChain lança *chains* lineares.
- **Jan/2024** – LangGraph 0.0.x: primeira versão pública.
- **Mai/2024** – LangGraph Studio + Cloud.
- **2025** – LangGraph 1.x estável com *durable execution*.

Comparativo

Framework	Ciclos	Persistência
LangGraph	Nativo	Checkpointers
CrewAI	Limitado	Memory mod.
AutoGen	Por chat	Externa
Swarm	Não	Não

O que funcionou

A combinação de StateGraph com aresta condicional deixou o roteamento declarativo e fácil de testar — dá pra escrever evals direto sobre o grafo compilado. Persistência em JSON foi suficiente para o protótipo, e a latência abaixo de 60 ms no P95 mantém o agente confortável em chat síncrono.

Limitações observadas

A classificação por keywords não captura semântica: a classe `geral` acaba absorvendo qualquer frase que não bate com as outras — daí o *precision* de 0,73. Mensagens multi-intenção, como “meu produto chegou quebrado e quero suporte”, são forçadas a uma única rota. Também não tem suporte a contexto profundo entre turnos.

Próximos passos

Trocar o `detect_intent` por um classificador LLM *few-shot* com saída estruturada via Pydantic; adicionar um nó de Human-in-the-Loop para casos em que o `confidence` fique baixo; e migrar a persistência JSON para um `SqliteSaver`, ganhando *durable execution*.

Referências principais (ABNT NBR 6023)

1. LANGCHAIN AI. **LangGraph – Overview and Graph API**. Documentação oficial, 2024. langchain-ai.github.io/langgraph/.
2. LANGCHAIN AI. **Persistence and Checkpointers**. 2024. langchain-ai.github.io/langgraph/concepts/persistence/.
3. LANGCHAIN AI. **Human-in-the-loop**. 2024. langchain-ai.github.io/langgraph/concepts/human_in_the_loop/.
4. WU, Q. *et al.* **AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation**. arXiv:2308.08155, 2023.
5. YAO, S. *et al.* **ReAct: Synergizing Reasoning and Acting in Language Models**. ICLR, 2023.
6. OPENAI. **Swarm**. GitHub, 2024.
7. LANGCHAIN AI. **LangGraph Academy**. academy.langchain.com/courses/intro-to-langgraph.

Repositório do grupo

github.com/Ceciliarufatto/langgraph
Código-fonte, dataset, métricas, painel e demo em GIF.

